

Desafío: Frontend Angular para microservicio Django listo para producción

Objetivo

Construir una SPA en Angular que consuma la API del microservicio Django del desafío backend, permitiendo gestionar Personas y/o Productos. La app debe estar preparada para correr en desarrollo y en un entorno cercano a producción (build optimizado, configuración por entorno, opcionalmente Docker).

Requisitos funcionales (MVP)

Pantallas mínimas:

- Listado de Personas
- Formulario de creación / edición de Persona
- Listado de Productos
- Formulario de creación / edición de Producto
- Layout básico con navegación entre secciones

Comportamiento esperado:

Personas:

- Listar Personas consumiendo GET /api/v1/persons/ (con paginación si la API la expone).
- Mostrar al menos: first_name, last_name, email, created_at.
- Permitir filtrar por email y last_name mediante inputs.
- Permitir ordenar por created_at (asc / desc) si la API lo soporta.
- Crear nueva persona con POST /api/v1/persons/.
- Editar persona con GET /api/v1/persons/{id}/ y PUT/PATCH /api/v1/persons/{id}/.
- Eliminar persona con DELETE /api/v1/persons/{id}/ (confirmación antes de borrar).

Productos:

- Listar Productos consumiendo GET /api/v1/products/.
- Mostrar al menos: name, sku, price, owner (si aplica), created_at.
- Permitir filtrar por sku y por rango de precio (price_min, price_max) usando inputs.
- Permitir búsqueda general por nombre (q) si la API lo soporta.
- Crear producto con POST /api/v1/products/.
- Editar producto con GET /api/v1/products/{id}/ y PUT/PATCH /api/v1/products/{id}/.
- Eliminar producto con DELETE /api/v1/products/{id}/ (confirmación).

Formularios:

- Usar formularios reactivos (FormGroup).
- Validaciones mínimas:
 - Persona: first_name y last_name requeridos (1–100), email requerido con formato válido.
 - Producto: name requerido (1–150), sku requerido (3–50), price requerido, numérico y ≥ 0 .
- Mostrar mensajes de error de forma clara debajo de los campos o en un bloque de errores.

Funcionalidades extra (opcional, suma puntos)

- Autenticación vía JWT contra el backend (pantalla de login, guard de rutas, interceptor).
- Manejo de usuario autenticado en la UI (mostrar nombre, opción de logout, etc.).
- Filtros avanzados (por ejemplo, combinación de varios filtros y búsqueda en tiempo real).
- Tabla con ordenamiento de columnas y paginación controlada desde el frontend.
- Select de Persona como owner de Producto, cargando opciones desde /api/v1/persons/.
- Pequeño indicador de estado del backend (por ejemplo, ping a /healthz o /readyz).

Requisitos no funcionales

- Angular 16+ y TypeScript.
- Uso de Angular Router con al menos estas rutas:
 - /persons (listado)
 - /persons/new y /persons/:id/edit

- /products (listado)
- /products/new y /products/:id/edit
- /login (solo si se implementa autenticación)
- Arquitectura recomendada para nivel semi-senior:
 - Módulos por dominio (ej. PersonsModule, ProductsModule, AuthModule).
 - Servicios para acceso a API (PersonService, ProductService, AuthService).
 - SharedModule con componentes reutilizables (inputs, tablas, etc.) si es necesario.
- Manejo de estado simple usando servicios + RxJS (no es obligatorio usar NgRx).
- Manejo de errores HTTP:
 - Interceptor que capture errores 4xx/5xx y muestre mensajes amigables.
 - Manejo especial de 401/403 si se implementa autenticación (redirigir a /login).
- Indicadores de carga (spinners o mensajes de "Cargando...") mientras se esperan respuestas.
- Mensajes para estados vacíos ("No se encontraron resultados").

Preparación para producción

- Uso de environments:
 - environment.ts (desarrollo) y environment.prod.ts (producción).
 - Definir al menos apiUrl (ej. http://localhost:8000) sin hardcodear URLs en componentes.
- Build de producción:
 - ng build --configuration production
- Se valora:
 - Configuración lista para servir el build con un servidor estático (por ejemplo Nginx).
 - Dockerfile opcional para construir y servir la app.

Entregables

- Repositorio Git (GitHub, GitLab o similar) con:
 - Código fuente de la app Angular.
 - README con:
 - Requisitos (Node, versión mínima de Angular, etc.).
 - Pasos para levantar el frontend en desarrollo.
 - Cómo configurar apiUrl para apuntar al backend del desafío Django.
 - Pasos para generar el build de producción.
 - (Opcional) Pasos para correr tests.
- No es necesario publicar la app en un hosting, pero si lo hace, incluir la URL en el README.

Requisitos técnicos / setup local

- Se puede asumir que el backend corre en http://localhost:8000.
- Se espera que el frontend maneje correctamente CORS (lado backend) y que no dependa de proxies poco claros.
- Incluir un ejemplo de configuración de environments, por ejemplo:

```
environment.ts (desarrollo):
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8000'
};
```

```
environment.prod.ts (producción):
export const environment = {
  production: true,
  apiUrl: 'https://api.midominio.com'
};
```

Nivel del desafío

- Pensado para un perfil fullstack :
 - Se espera autonomía configurando Angular, servicios HTTP y routing.
 - Buen criterio para separar responsabilidades entre componentes y servicios.

- Cuidado en validaciones de formularios y manejo de errores.
- No es obligatorio usar librerías avanzadas de estado (NgRx), pero se valora si se usan bien.

Criterios de evaluación (100 pts)

- Correctitud funcional (listados, formularios, integración real con la API) – 35 pts
- Calidad de código y arquitectura (módulos, servicios, separación de responsabilidades) – 25 pts
- UX básica y manejo de errores (validaciones visibles, loaders, mensajes claros) – 15 pts
- Preparación para producción (environments, build, opcionalmente Docker) – 10 pts
- Pruebas y calidad mínima (al menos algunos unit tests y uso de lint) – 10 pts
- Bonus: autenticación JWT, E2E básicos, integración suave con el desafío Django (docker-compose, documentación extra) – hasta +5 pts